```
import streamlit as st
from ollama import ChatResponse
from ollama import chat
import json
import fitz # pymupdf
import re
import pandas as pd
import base64

st.set_page_config(page_title="Resume Analyzer", page_icon=":bookmark_tabs:",
layout="wide")
st.title("AI Resume Analyzer")
st.markdown("**Upload your resume and get a summary, key skills score and
improvment suggestions**")

uploaded_file = st.file_uploader("Upload your resume", type=["pdf"])

if uploaded_file:
    uploaded_file.seek(0)
    pdf = fitz.open(stream=uploaded_file.read(), filetype="pdf")
    text = ""
    for page in pdf: # pdf.pages:
        page_text = page.get_text() # extract_text()
        if page_text:
            text += page_text + "\n"

    text_clean = re.sub(r"(?<!\n)\n(?!\n)", " ", text)
    col1,col2 = st.columns([1, 1])

    with col1:
        st.subheader("Resume Preview")
        base64_pdf = base64.b64encode(uploaded_file.getvalue()).decode('utf-8')
        pdf_display = F'<iframe src="data:application/pdf;base64,{base64_pdf}"
width="800" height="1000" type="application/pdf"></iframe>'
        st.markdown(pdf_display, unsafe_allow_html=True)

    with col2:
```

```
st.subheader("AI analysis")
if st.button("Analyze Resume"):
    with st.spinner("Analyzing resume..."):
        prompt = f"""
                Extract and analyze this resume. if it is not a resume tell the user
that it is not a resume but what you think it is and not need to return any
information.

                if it is a resume return:
                1. Basic information:
                   - Name
                   - Email
                   - Phone
                   - Address
                   - Social media handles
                2. Short summary
                3. Key skills
                4. Improvement suggestions
                5. Score out of 100 and explain why based on:
                   - Skills (30 points)
                   - Experience and acheivement (30 points)
                   - Clarity and formatting (20 points)
                   - Overall impression (20 points)

                Also return:
                Score JSON:
                {{"skills": X, "experience": X, "clarity": X, "overall": X}}

                Resume:
                {text_clean}
                """

        response: ChatResponse =  chat(
            model="llama3.2:latest",
            messages=[{"role": "user", "content": prompt},]
        )
```

```
result = response["message"]["content"]

parts = result.split("Score JSON:")
analysis_text = parts[0]
st.write(analysis_text)

if len(parts) > 1:
    try:
        match = re.search(r"\{.*\}", result, re.DOTALL)

        if match:
            score_data = json.loads(match.group())

            st.subheader("Score Breakdown")

            df = pd.DataFrame({
                "Category": list(score_data.keys()),
                "Score": list(score_data.values())
            })

            st.bar_chart(df.set_index("Category"))

            st.subheader("Overall Score")
            total_score = sum(score_data.values())
            st.progress(min(total_score / 100, 1.0))
        else:
            st.warning("No JSON found in model response")

    except Exception as e:
        st.warning(f"Could not parse score JSON: {e}")
```